Centro Universitario Guadalajara

Lic. En Inteligencia Artificial y Ciencia de Datos

Programación IV

Investigación Teórica: Conceptos Clave de JSON, Diagramas ER y la Librería Requests en Python

Autor:
Patricia Jaime Pérez

Profesor:
Diego Alberto Ayala Vázquez

2 de septiembre de 2026

Índice

1. Introducción **2**

2. Serialización y Deserialización de JSON **2**

2.1. Definición y Propósito 2

2.2. Proceso en Python 2

3. Tipos de Datos Soportados en JSON **3**

3.1. Tipos de Datos Oficiales 3

3.2. Mapeo de Tipos: JSON vs. Python 4

4. Diagramas Entidad-Relación (ER) **4**

4.1. Concepto y Propósito 4

4.2. Componentes Principales 4

4.3. Modelado de Ejemplo 5

5. Documentación de la Librería `requests` en Python **5**

5.1. Descripción General 5

5.2. Métodos Principales 5

5.3. Atributos y Métodos de la Respuesta (`Response`) 6

5.4. Ejemplo de Uso Práctico 6

6. Conclusiones **7**

1. Introducción

En el desarrollo de software moderno y la integración de sistemas, el intercambio eficiente de datos y el diseño estructurado de la información constituyen los pilares fundamentales de la arquitectura Cliente-Servidor. La estandarización de formatos como JSON (*JavaScript Object Notation*) permite que aplicaciones construidas sobre distintas tecnologías se comuniquen de manera transparente.

El presente trabajo aborda los conceptos teóricos y prácticos fundamentales previos al desarrollo de servicios web y APIs. En primer lugar, se analiza el proceso de serialización y deserialización de datos, así como los tipos de datos nativos soportados por el estándar JSON[cite: 1]. Posteriormente, se examinan los Diagramas Entidad-Relación (ER) como herramienta esencial para el modelado de bases de datos[cite: 1]. Finalmente, se documenta el uso y funcionamiento de la librería `requests` de Python para el consumo programático de servicios HTTP[cite: 1].

2. Serialización y Deserialización de JSON

2.1. Definición y Propósito

En la programación orientada a objetos y en el desarrollo web, las estructuras de datos residen en la memoria RAM del sistema en formatos complejos y específicos del lenguaje de programación. Para transmitir estas estructuras a través de una red o almacenarlas en un archivo, es necesario convertirlas en una secuencia de bytes o en una cadena de texto.

- **Serialización (Marshaling):** Es el proceso de transformar un objeto o estructura de datos en memoria (por ejemplo, un diccionario en Python o una instancia de clase) a un formato estandarizado en texto plano (como una cadena JSON) que pueda ser transmitido por la red mediante HTTP o guardado en disco[cite: 1].
- **Deserialización (Unmarshaling):** Es el proceso inverso, mediante el cual se toma una cadena de texto en formato JSON recibida de una fuente externa y se reconstruye en una estructura de datos o un objeto nativo de la memoria del lenguaje de programación receptor[cite: 1].

2.2. Proceso en Python

En Python, el módulo nativo `json` proporciona funciones principales para realizar ambas operaciones:

`json.dumps()`: Serializa un objeto de Python a una cadena de texto en formato JSON.

`json.loads()`: Deserializa una cadena de texto JSON a un objeto nativo de Python (diccionarios, listas, etc.)[cite: 1].

```python
import json

# Datos en memoria (Diccionario de Python)
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "activo": True
}

# 1. SERIALIZACI N: Diccionario Python -> Cadena JSON
json_string = json.dumps(estudiante, indent=4)
print("Cadena JSON serializada:")
print(json_string)

# 2. DESERIALIZACI N: Cadena JSON -> Diccionario Python
datos_recibidos = json.loads(json_string)
print("\nObjeto Python reconstruido:")
print(f"Nombre: {datos_recibidos['nombre']}, Tipo: {type(datos_recibidos)}
    ")
```

Listing 1: Ejemplo de Serialización y Deserialización en Python

3. Tipos de Datos Soportados en JSON

El estándar JSON (*RFC 8259*) define una sintaxis liviana para el intercambio de datos que soporta únicamente un conjunto acotado de tipos de datos primitivos y estructurados[cite: 1].

3.1. Tipos de Datos Oficiales

JSON admite exactamente los siguientes seis tipos de datos[cite: 1]:

1. **Cadena (*String*):** Secuencia de caracteres Unicode delimitada por comillas dobles (`"..."`)[cite: 1].

2. **Número (*Number*):** Valores numéricos enteros o de punto flotante en notación decimal o científica[cite: 1]. No distingue entre enteros y flotantes.

3. **Booleano (*Boolean*):** Valores de lógica binaria representados únicamente por las palabras clave en minúscula `true` o `false`[cite: 1].

4. **Objeto (*Object*):** Colección no ordenada de pares clave-valor delimitada por llaves ({})[cite: 1]. Las claves deben ser obligatoriamente cadenas de texto.

5. **Arreglo / Lista (*Array*):** Colección ordenada de valores delimitada por corchetes (`[...]`)[cite: 1]. Los elementos pueden ser de cualquier tipo de dato válido.

6. **Nulo (*Null*):** Representa la ausencia explícita de valor utilizando el literal `null`[cite: 1].

3.2. Mapeo de Tipos: JSON vs. Python

A continuación se presenta una tabla comparativa entre los tipos de datos de JSON y sus correspondientes equivalencias en el lenguaje Python:

Tipo de Dato JSON	Equivalente en Python	Ejemplo Sintáctico JSON
String	`str`	`."na"`[cite: 1]
Number	`int` / `float`	`20` / `95.5`[cite: 1]
Boolean	`bool`	`true` / `false`[cite: 1]
Object	`dict`	`{"id": 1, "rol": ."dmin"}`[cite: 1]
Array	`list`	`["Git", "Python"]`[cite: 1]
Null	`None`	`null`[cite: 1]

Tabla 1: Mapeo entre tipos de datos JSON y estructuras de datos de Python.

4. Diagramas Entidad-Relación (ER)

4.1. Concepto y Propósito

Un **Diagrama Entidad-Relación (ER)** es un modelo conceptual utilizado en la ingeniería de software y el diseño de bases de datos relacionales[cite: 1]. Permite visualizar la estructura lógica de un sistema, identificando los objetos del dominio (entidades), sus características (atributos) y la manera en que se vinculan entre sí (relaciones)[cite: 1].

4.2. Componentes Principales

- **Entidad:** Representa un objeto real o abstracto del dominio sobre el cual se almacenan datos (ej. *Estudiante*, *Curso*)[cite: 1]. Se dibuja con un rectángulo.

- **Atributo:** Propiedad o característica que describe a una entidad (ej. *nombre*, *correo*)[cite: 1]. Se representa con un óvalo o elipse.
- **Relación:** Describe la asociación lógica entre dos o más entidades. Se representa con un rombo.
- **Clave Primaria (PK):** Atributo único que identifica de forma inequívoca a cada registro de una entidad.

4.3. Modelado de Ejemplo

A continuación se presenta un diagrama conceptual ER representando la relación entre las entidades **Estudiante** y **Curso**:

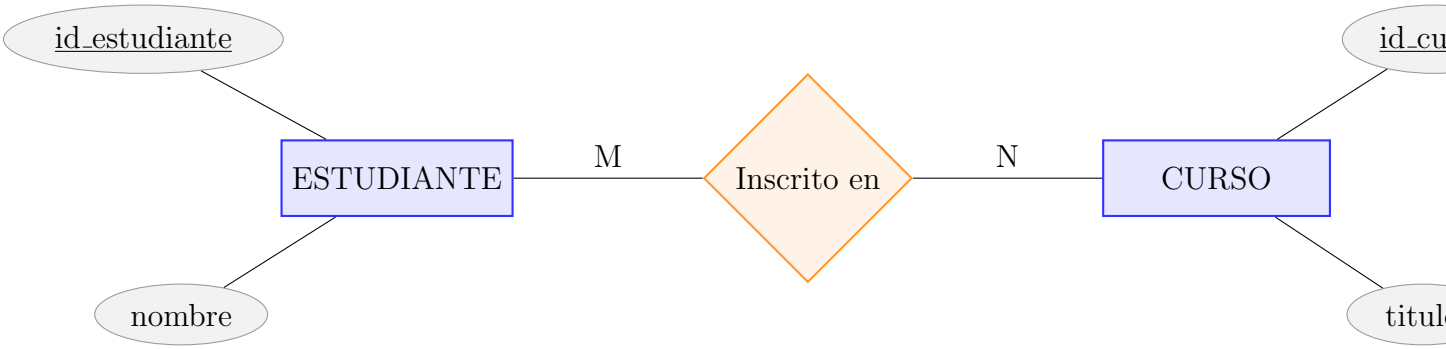


Figura 1: Diagrama ER simplificado de la relación N:M entre Estudiantes y Cursos.

5. Documentación de la Librería `requests` en Python

5.1. Descripción General

`requests` es una de las librerías de terceros más populares en Python para realizar peticiones HTTP de forma sencilla e intuitiva[cite: 1]. Abstrae la complejidad de la biblioteca nativa `urllib3`, permitiendo a los desarrolladores enviar solicitudes `GET`, `POST`, `PUT`, `DELETE`, manejar autenticación y gestionar parámetros sin requerir configuración manual compleja[cite: 1].

5.2. Métodos Principales

- `requests.get(url, params=...)`: Realiza una petición de lectura para obtener datos del servidor[cite: 1].

- `requests.post(url, json=..., data=...)`: Envía datos en el cuerpo del mensaje para crear recursos[cite: 1].
- `requests.put(url, json=...)`: Reemplaza un recurso existente[cite: 1].
- `requests.delete(url)`: Elimina un recurso del servidor[cite: 1].

5.3. Atributos y Métodos de la Respuesta (`Response`)

Cuando se realiza una petición, la librería devuelve un objeto de la clase `Response` con las siguientes propiedades destacadas:

- `response.status_code`: Retorna el código de estado HTTP en formato numérico (ej. 200, 404, 500)[cite: 1].
- `response.json()`: Método que realiza la deserialización automática del cuerpo de la respuesta en formato JSON a estructuras nativas de Python (diccionarios o listas)[cite: 1].
- `response.text`: Contenido completo de la respuesta interpretado como cadena de texto.
- `response.headers`: Diccionario que contiene las cabeceras recibidas del servidor.

5.4. Ejemplo de Uso Práctico

El siguiente script en Python ilustra cómo consumir un endpoint público consultando información y manejando la respuesta mediante `requests`[cite: 1]:

```
import requests

# URL base del servicio web
URL = "https://jsonplaceholder.typicode.com/users"

try:
    # 1. Enviar peticion HTTP GET
    response = requests.get(URL, timeout=5)

    # 2. Verificar el codigo de estado HTTP
    if response.status_code == 200:
        print("Petici n exitosa (200 OK)")

        # 3. Deserializar respuesta JSON
```

```
        usuarios = response.json()

        # Iterar sobre los primeros 3 usuarios
        for usuario in usuarios[:3]:
            print(f"ID: {usuario['id']} | Nombre: {usuario['name']} |
    Email: {usuario['email']}")
    else:
        print(f"Error en la solicitud: C digo {response.status_code}")

except requests.exceptions.RequestException as e:
    print(f"Error de conexi n con el servidor: {e}")
```

Listing 2: Consumo de API REST utilizando la librería requests en Python

6. Conclusiones

El estudio formal de estos conceptos teóricos permite comprender la base del desarrollo de software distribuido[cite: 1]:

1. La **serialización y deserialización** son procesos indispensables para puenteat la brecha entre los datos en memoria del programa y la transmisión en redes informáticas[cite: 1].

2. El formato **JSON** se afianza como el estándar universal debido a su sintaxis reducida y su compatibilidad directa con los tipos de datos fundamentales de la mayoría de los lenguajes de programación[cite: 1].

3. Los **Diagramas Entidad-Relación** garantizan un diseño estructurado y coherente de los modelos de datos antes de la implementación en sistemas de gestión de bases de datos[cite: 1].

4. La librería **`requests`** simplifica significativamente la creación de clientes HTTP en Python, facilitando la integración con servicios externos y sentando las bases para construir y consumir APIs robustas[cite: 1].

Referencias

[1] ECMA International / IETF, *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, 2017.

[2] Reitz, K. et al., *Requests: HTTP for Humans (Official Documentation)*, https://requests.readthedocs.io/, 2023.

[3] Silberschatz, A., Korth, H. F., & Sudarshan, S., *Fundamentos de Bases de Datos*, 6ta Edición, McGraw-Hill, 2014.

[4] Ayala Vázquez, Diego Alberto, *Material de Clase: Integración de Sistemas y Consumo de APIs*, Documentación docente, 2026[cite: 1].